



Cátedra Tecnología de Sistemas

Programación Avanzada con C#

Código: 00830

Proyecto 1. Valor 1%

Temas de Estudio

1. Tema 1 Introducción al lenguaje de programación
2. Tema 2 Particularidades del lenguaje C#
3. Tema 3 Manejo de excepciones en C#
4. Tema 4 Conceptos de Interfaz Gráfica

Objetivo

Aplicar los conceptos iniciales del curso como interfaz gráfica y manejo de excepciones, en el desarrollo de una aplicación práctica que sirva como base para proyectos más avanzados durante el cuatrimestre.

Software de Desarrollo

Visual Studio Community 2022 (C# Net 8.0)

Desarrollo

La empresa MITICAX dedicada al desarrollo de videojuegos, le ha contratado para crear un juego de batallas con criaturas místicas. Este será el primer proyecto del curso y servirá como base para desarrollos más complejos en las siguientes etapas.

La aplicación debe desarrollarse en C# en un entorno visual y se requiere que contenga las siguientes funcionalidades:

- Registrar y consultar Criaturas
- Registrar y consultar Jugadores
- Registrar y consultar Inventario de Criaturas del Jugador
- Registrar y consultar Batallas
- Registrar y consultar Equipos

- Registrar y consultar Rondas
- Consultar el top 10 de ganadores

Registrar y consultar criaturas

Esta opción permitirá al jugador registrar y consultar los datos de las distintas criaturas disponibles para las batallas. Estas criaturas conformarán el catálogo que se utilizará durante el juego.

Al registrar una criatura, se deben ingresar los siguientes datos:

- IdCriatura (int), identificador único de la criatura. Debe validarse que no existan valores repetidos.
- Nombre (string), nombre de la criatura.
- Tipo (string), puede ser agua, tierra, aire o fuego.
- Nivel (int), Nivel de criatura, con las siguiente categorías :
 - ☐ 01-Iniciado
 - ☐ 02-Aprendiz
 - ☐ 03-Estudiante
 - ☐ 04-Avanzado
 - ☐ 05-Maestro
- Poder (int), Define el nivel de fuerza de la criatura.
- Resistencia (int), indica el nivel de resistencia ante los ataques.
- Costo (int), Representa el valor de la criatura en la tienda del juego. La moneda utilizada son los Cristales, por ejemplo, un costo de 10 significa 10 cristales. El costo debe establecerse según el siguiente rango:

NIVEL CRIATURA	Rango de Costo
Iniciado	< 100
Aprendiz	>=100 y < 300
Estudiante	>=300 y < 600
Avanzado	>=600 y < 1200
Maestro	>=1200

- Todos los datos son obligatorios.
- Una vez registrada la información de una criatura, debe agregarse a un arreglo de objetos de la clase "CriaturasEntidad".

- El arreglo debe de ser de 20 posiciones.
- La interfaz debe incluir un control “DataGridView” que muestre todos los datos de todas las criaturas registradas.
- Cada vez que se registre una nueva criatura, esta debe reflejarse automáticamente en el DataGridView y cuando un campo registrado tenga una descripción, en la tabla se debe mostrar la descripción no el valor. El DataGridView debe mostrar todos los datos de la entidad.

Registrar y consultar jugadores

Está opción permitirá registrar y consultar los datos de los jugadores del juego. Para esta funcionalidad se debe crear una clase denominada “JugadorEntidad”.

Al registrar un jugador se deben ingresar los siguientes datos:

- IdJugador (int), identificador único del jugador. Se debe validar que no existan valores repetidos.
 - Nombre (string), nombre del jugador.
 - FechaNacimiento (DateTime), fecha de nacimiento del jugador. Se debe validar que tenga más de 10 años al momento del registro.
 - Nivel (int), Indica el nivel de experiencia del jugador.
 - Al registrarse, el jugador inicia en el nivel 01-Novato
 - Al ganar 5 batallas asciende a Estudiante
 - Al ganar 10 batallas asciende a Maestro
 - Al ganar 20 batallas asciende a Supremo
 - Cristales (int), Moneda en el juego. Todo jugador inicia con 100 Cristales.
 - BatallasGanadas (int), Contador de batallas ganadas. Todos los jugadores inician en cero. Este dato solo se actualiza al ganar una batalla.
-
- Todos los datos son obligatorios.
 - Los jugadores deben almacenarse en un arreglo de objetos de la clase “JugadorEntidad”. El arreglo debe de ser de 20 posiciones.
 - La interfaz debe incluir un control “DataGridView” que muestre la lista de jugadores registrados.
 - Cada vez que se agrega un nuevo jugador, este debe reflejarse automáticamente en el DataGridView y cuando un campo registrado tenga una descripción, en la tabla se debe mostrar la descripción no el valor. El DataGridView debe mostrar todos los datos de la entidad.

Registrar y consultar inventario de criaturas de jugador

Esta opción permitirá al jugador adquirir criaturas para su inventario personal.

- Al seleccionar una criatura, el sistema debe validar si el jugador posee la cantidad suficiente de Cristales para comprarla.
- Si cumple con el requisito, se permite agregar la criatura al inventario, si no, se debe enviar un mensaje que: “No posee la cantidad de cristales suficientes para obtener la criatura”.
- Si compra la criatura se debe descontar del saldo del usuario el costo de la criatura.

Para este proceso se debe crear una clase llamada “InventarioJugadorEntidad”.

Se debe poder registrar la siguiente información:

- IdJugador (int), identificador único del jugador. Se debe validar en el sistema que no existan valores repetidos. IdCriatura (int), Identificador de criatura adquirida. Se debe validar que esa id no se repita, es decir, el jugador no puede tener más de una criatura con el mismo IdCriatura. Poder (int), Nivel de fuerza de la criatura. Al momento de la compra adopta el valor original y conforme el usuario **gane rondas** irá aumentando el poder en 5.
- Resistencia (int), Nivel de resistencia ante los ataques. Al momento de la compra adopta el valor original pero conforme se pierdan batallas irá disminuyendo.
 - Todos los datos son obligatorios.
 - Una vez capturada la información se debe agregar a un arreglo de objetos de tipo “InventarioJugadorEntidad”. El arreglo debe de ser de 30 posiciones.
 - La interfaz debe mostrar un control “DataGridView” que muestre todo el inventario de criaturas.(Debería especi)
 - Cada vez que se agregue una nueva criatura al inventario, debe reflejarse automáticamente en el DataGridView y cuando un campo registrado tenga una descripción, en la tabla se debe mostrar la descripción no el valor. El DataGridView debe mostrar todos los datos de la entidad.

Registrar y consultar Equipos

Esta opción permitirá al jugador registrar el equipo de criaturas que utilizará en las batallas. Para esta funcionalidad se debe crear una clase llamada “EquipoEntidad”.

Se debe registrar la siguiente información:

-
- IdEquipo (int), Identificador único del equipo. Se debe validar en el sistema que no existan valores repetidos
- IdJugador (int), Identificador del jugador al que pertenece el equipo.

- IdCriatura1 (int), Identificador de la primera criatura seleccionada que debe estar registrada previamente.
- IdCriatura2 (int), Identificador de la segunda criatura seleccionada que debe estar registrada previamente.
- IdCriatura3 (int), Identificador de la tercera criatura seleccionada que debe estar registrada previamente.

Los IdJugador, IdBatalla y IdCriatura deberían ser objetos del tipo que correspondan.

- Todos los datos son obligatorios.
- Una vez capturada la información se debe agregar a un arreglo de objetos de tipo "EquipoEntidad". El arreglo debe de ser de 40 posiciones.
- Se deberá mostrar un control "DataGridView" que muestre todos los Equipos del jugador. Cuando se agrega un nuevo equipo deberá verse reflejado de manera automática en el DataGridView.

Registrar y consultar Batalla

Esta opción permitirá iniciar y consultar batallas entre dos jugadores. Cada jugador seleccionará un equipo de criaturas previamente registrado y dichas criaturas se enfrentarán para determinar el ganador de la batalla.

Para esta funcionalidad se debe crear una clase llamada "BatallaEntidad".

Se debe poder registrar los siguientes datos :

- IdBatalla (int), identificador único de la batalla. Se debe validar en el sistema que no existan valores repetidos.
- IdJugador1 (int), Identificador del primer jugador en la batalla, que debe ser un jugador ya registrado.
- IdEquipo1 (int), Identificador del equipo del primer jugador que debe estar registrado previamente.
- IdJugador2 (int), Identificador del segundo jugador en la batalla, que debe ser un jugador ya registrado.
- IdEquipo2 (int), Identificado del equipo del segundo jugador que debe estar registrado previamente. Ganador (int), Almacena el identificador del jugador que gana la batalla.
- Fecha (date), Almacena la fecha en que se realiza la batalla.

Los IdJugador y IdEquipo deberían ser objetos del tipo que correspondan.

- Los datos iniciales que se registran es el código de batalla, jugador 1 y 2 y Equipo1 y 2. Una vez capturada la información se debe almacenar en un arreglo de objetos de tipo "BatallaEntidad".

- El arreglo debe de ser de 50 posiciones.
- Cuando un jugador gana, también se debe actualizar el dato en la entidad de jugador, en incrementar en uno el campo batallasGanadas.
- El jugador puede escoger cualquier equipo registrado a su nombre para la batalla.
- Se deberá mostrar un control “DataGridView” que muestre todas las batallas. Cuando se agrega una nueva batalla deberá verse reflejada de manera automática en el DataGridView.

Registrar y consultar Rondas

Esta funcionalidad permite registrar el desarrollo de las rondas de cada batalla. Para ello se debe crear una clase llamada “RondasEntidad”.

En esta opción debe registrar la siguiente información:

- IdRonda (int), Determina el número de la ronda realizada en la batalla. Los únicos valores permitidos son 1, 2 y 3.
- IdBatalla (int), Identificador de la batalla que se está realizando.
- IdJugador1 (int), Identificador del primer jugador al que pertenece el equipo.
- IdCriatura1 (int), Identificador de la primera criatura seleccionada.
- IdJugador2 (int), Identificador del segundo jugador al que pertenece el equipo.
- IdCriatura2 (int), Identificador de la segunda criatura seleccionada.
- GanadorRonda (int), Identificador del jugador que gana la ronda.

Todos los datos son obligatorios.

La ronda se debe almacenar en un arreglo de objetos de tipo RondasEntidad de 100 posiciones.

El proceso de rondas **será automático**, es decir, que es el programa quien define la ronda y registra los datos, basado en el registro de la batalla. En esta parte el programa seleccionará aleatoriamente una criatura de cada equipo del jugador (No se puede repetir criatura del jugador en las distintas rondas) y los pondrá a “competir”.

El ganador la ronda recibe 10 Cristales.

El ganador de la batalla recibe 30 Cristales adicionales.

La criatura que gana la ronda aumenta su poder en 5 puntos dentro del inventario del jugador.

Criterios para determinar el ganador de una ronda

1. Cada criatura tiene un valor de poder y otro de resistencia.
2. La aplicación debe asignar de forma aleatoria quién ataca primero.
3. Entonces el primero en atacar envía su poder contra la resistencia del oponente, si el poder del atacante supera la resistencia del contrincante, entonces el primero gana, esto es, si : $resistencia - poder \leq 0$.
4. Si no se supera la resistencia, es decir, si la resta de resistencia menos poder no es menor o igual a cero, entonces hay contraataque, es decir, se mide el poder del segundo contra la resistencia del primero y de igual manera si el poder supera la resistencia, el segundo sería el ganador.
5. Si esto tampoco sucede entonces, para determinar el ganador de la ronda se hará lo siguiente: Se toma el valor del poder del primero y se le resta a la resistencia del segundo, el resultado es el remanente de poder. Se hace lo mismo con el segundo se toma el poder y se le resta la resistencia del primero.
6. Entonces, si el remanente de poder del primero es mayor al del segundo, el primero gana y viceversa.
7. Si los valores llegaran a ser iguales entonces el ganador se determinará de manera aleatoria.
8. Siempre debe existir un ganador en cada ronda.

¿Quién gana la batalla?

¡Gana la batalla el jugador que obtenga dos o más rondas ganadas!

Ejemplo de batalla

Supongamos que la criatura uno tiene poder 10 y resistencia 11. La Criatura 2 tiene poder 9 y resistencia 6.

Para la primera batalla la aplicación selecciona el atacante, supongamos que el atacante es la criatura uno entonces, se toma el poder de la criatura 1 y la resistencia de la criatura 2. Al valor de resistencia de la criatura 2 le restamos el poder de la criatura 1, esto es $6 - 10$, el resultado sería -4 y esto indica que la criatura 1 gana la ronda.

Ahora supongamos que la aplicación seleccionó la criatura 2 como atacante en la primera ronda. Se hace la misma operación la resistencia de la criatura uno menos el poder de la criatura 2, esto es $11 - 9$, el resultado es 2, este valor no es menor que 0 entonces hay contraataque que sería la operación que hicimos arriba resistencia de la criatura 2 menos poder de la criatura 1, esto es $6 - 10$ que daría el gane a la criatura 1.

Pensemos ahora que ni en el ataque, ni en el contraataque ninguna operación resultó menor o igual a 0, entonces el siguiente paso es determinar el remanente de poder, esto es tomar el poder menos la resistencia del contrincante. Con los datos que pusimos arriba esto seía.:

Remanente de poder de la Criatura 1 es 10 que es su poder menos 6 de resistencia del contrincante entonces su remanente de poder es 4. Para la Criatura 2 sería su poder que es 9 menos 11, esto nos da un valor menor que 0. Para determinar el ganador comparamos los remanentes el mayor, gana a ronda, en este caso la criatura 1 gana.

Consultar el top 10 de ganadores

Esta opción permite visualizar en pantalla los 10 jugadores con más batallas ganadas. Para tal fin debe utilizar un control “DataGridView”. El control debe mostrar todas las propiedades de la clase Jugador. (6 columnas)

Consideraciones técnicas

Uso de comentarios

- Es obligatorio implementar comentarios en el código de programación para documentar adecuadamente el proyecto.
- En la parte superior de cada archivo (.cs)-- clases, formularios, programas principales—debe incluirse la siguiente información:
- Universidad- Cuatrimestre
- Nombre y descripción del proyecto
- Nombre del estudiante
- Fecha de realización del código
- Además, dentro del código se deben agregar comentarios claros que expliquen el propósito de los métodos, propiedades y procesos importantes.

Ejemplo:


```
1  /**
2   * UNED X Cuatrimestre
3   * Proyecto X: Descripción del proyecto
4   * Estudiante:
5   * Fecha:
6   */
7  namespace CapaEntidades
8  {
```

El proyecto debe resolverse aplicando principios de Programación orientada a objetos (POO).

Las clases de objetos actúan como entidades:

- No deben contener lógica para solicitar información a la persona usuaria, solo debe tener la estructura de la clase y sus propiedades.
- El nombre del objeto y las propiedades no deben de llevar tilde ni ñ.

Organización en capas:

- La solución debe Implementarse utilizando **proyectos independientes** con el fin de tener las distintas capas recomendadas: Capa de Entidades, Capa de Acceso a Datos, Capa de Lógica Negocio, Capa de Presentación (Interfaz del Usuario), etc.
- Cada clase debe ser creada en archivo independiente (.cs).
- Las clases no deben tener métodos vacíos.
- Utilizar nombres significativos y consistentes para sus variables, métodos y propiedades.

Uso de combobox

- Los Combobox deben configurarse para que el jugador no pueda editar o escribir en él, solo debe permitir seleccionar una opción de la lista predefinida.

Uso de arreglos

- No deben utilizarse arreglos de tipo "int" ni "string", deben ser arreglos de objetos de las clases entidades definidas.
- Cada vez que se agrega un objeto al arreglo se deben limpiar los campos de la pantalla y mostrar el siguiente mensaje: ***"El registro se ha ingresado correctamente"***. Esto con el fin de que el usuario pueda ingresar otro registro.
- En caso de que el arreglo llegue a su límite y no se puedan ingresar nuevos registros, se debe visualizar un mensaje ***"No se pueden ingresar más registros"***.
- **Está prohibido el uso de colecciones (System.Collections o LINQ), solo se permite el uso de arreglos para este Proyecto.**
- Los arreglos deben ubicarse en la capa de Acceso a Datos.

Debe implementar el manejo de excepciones

- Si ocurre una excepción, el sistema no debe cerrarse
- Se debe mostrar un mensaje claro al jugador e implementar un manejo adecuado de la excepción para permitir continuar la ejecución.

Interfaz de usuario

- Debe usar interfaz de usuario con formularios GUI (Proyecto Windows Form).
- Se debe cuidar el orden de los formularios y el orden de lectura de los campos.
- Se debe cuidar la ortografía en los textos visibles al jugador.
- Se debe crear un menú principal para acceder a las opciones del sistema.

Uso de DataGridView

- Los títulos de las columnas deben establecerse previamente para que se muestren de forma correcta, y no deben dejar tomar el nombre de la propiedad de la clase.
- Los DataGridView deben configurarse como solo lectura, no se permite modificar los datos que se muestren desde el control. Tampoco debe permitir agregar o borrar nuevos registros.
- Cuando se seleccione un dato del DataGridView se debe seleccionar toda la fila.

Honestidad Académica

<https://audiovisuales.uned.ac.cr/play/player/23048>

Nota importante

- Cada estudiante es responsable del contenido que entrega.
- Si el archivo enviado no corresponde al proyecto correcto, no se aceptarán entregas posteriores fuera de la fecha establecida.
- Si se detecta que el contenido del archivo coincide con otro estudiante o no es de su autoría, se aplicarán las sanciones indicadas en el documento de Lineamientos ante casos de plagio disponible en la plataforma.

Indicaciones Importantes

- Es obligatorio que incluya todo el directorio donde se encuentra el Proyecto # 1.
- El Proyecto #1 debe desarrollarse en **Visual Studio Community 2022** que es la herramienta oficial del curso.
- Los trabajos deben realizarse en forma individual.
- Dentro del código del programa debe de indicar la documentación que explique cómo fue realizado el programa.
- Si utiliza código de algún ejemplo del libro, del algún profesor o de otra fuente que no sea de su autoría, debe de indicarlo.
- Comprima todos los archivos en un solo archivo .zip o .rar.
- **Nombre del archivo que envía:** debe ser nombre y primer apellido del estudiante, y nombre de la tarea. **Ejemplo: JuanRojas-Proyecto1.**
- La entrega del proyecto debe ser en las fechas establecidas en la plataforma de aprendizaje en línea Moodle, en el apartado que se indique.
- Si no concluyó a tiempo el proyecto, debe entregar lo que pudo hacer e incluir una carta explicando las razones por las cuales no finalizó.

Rúbrica de Evaluación

Criterio	Cumple a satisfacción lo indicado en la evaluación	Cumple medianamente en lo indicado en la evaluación	Cumple en contenido y formato, pero los aportes no son significantes	No cumple o no presenta lo solicitado
Presentación del código: Correcto uso de variables, nombres de métodos y variables significativos, no presenta métodos vacíos, se cuida de la ortografía y redacción en los mensajes de pantalla e internos.	5	3	2	0
Entrega la carpeta de la solución completa con todos los proyectos de cada capa. Indica dentro del programa en los comentarios, si se utiliza código de algún ejemplo del libro o algún profesor o de otra fuente que no sea de autoría del estudiante. El nombre del archivo es el solicitado. Agrega los comentarios de información del proyecto en las clases.	5	3	2	0
POO y creación de las clases (CriaturasEntidad, JugadorEntidad, InventarioJugadorEntidad, BatallaEntidad, EquipoEntidad, RondasEntidad). Utiliza propiedades y no atributos. No agrega constructores ni métodos adicionales. El proyecto está dividido en capas (Capa presentación, lógica de negocios, acceso a datos y entidades).	10	7	3	0
Registrar y consultar Criaturas	10	7	3	0
Registrar y consultar Jugadores	10	7	3	0
Registrar y consultar Inventario de Criaturas de Jugador.	10	7	3	0
Registrar y consultar Batalla.	10	7	3	0

Registrar y consultar Equipos.	10	7	3	0
Registrar y consultar Rondas.	10	7	3	0
Consultar top 10 de ganadores.	5	3	2	0
Correcto uso del manejo de excepciones con try catch.	5	3	2	0
Interfaz de usuario es fácil de usarse y presenta buen diseño. Presenta correctamente los DataGridView, los Combobox y los DateTimePicker. Muestra mensajes de error y de satisfacción de forma amigable. Presenta una correcta forma de navegación entre los formularios.	10	7	3	0
TOTAL	100			